

Time-Off Service — How It Works (Plain-English Guide)

A short, simple explanation of what this service does, the problems it solves, and how it works. Anything technical is explained in everyday words.

In one minute

- Employees ask for time off. Managers approve or reject.
- The **official** record of how many days everyone has lives in a separate company system called the **HCM** (think of it as the company's master HR system).
- Our service sits in front of the HCM. It gives employees and managers an **instant** answer, then quietly keeps everything in step with the HCM in the background — even if the HCM is slow or temporarily down.

A few words we use

Word	What it means (plain)
HCM	The company's main HR system. It is the <i>single source of truth</i> for how many days you have.
Balance	How many days you have.
Hold (pending)	When you ask for days, we set them aside ("on hold") so they can't be used twice — like a hotel putting a hold on your card before check-in.
Filing	Officially telling the HCM "this person used these days."
Debit / Credit	A debit removes days (a deduction); a credit adds days back (a refund).
Reconcile / Sync	Comparing our copy of a balance with the HCM's copy and fixing any difference.
Ledger	An unchangeable history log of every change to a balance.

The problem (why this is harder than it looks)

Two systems track the **same** numbers:

1. **Our service** — fast, talks to employees and managers.
2. **The HCM** — the official record, but slower and not always reachable.

They can easily drift apart, because:

- The HCM can change someone's days **on its own** — a work-anniversary bonus, or a start-of-year reset.
- The HCM might be **down** exactly when we need it.
- The HCM is *supposed* to reject impossible requests, but we are told **we can't rely on that**.

So our job is to always give users a correct, instant answer **and** never let the two systems disagree for long.

The big idea: "hold now, file later"

Example — Alice has 10 days:

1. Alice asks for **3 days**. We instantly set those 3 days *on hold*. Her available balance now shows **7**, and she gets an immediate "yes." (We did **not** wait for the HCM.)
2. Her manager **approves**. We keep the 3 days on hold.
3. In the background — usually within about 2 seconds — we tell the HCM "Alice used 3 days." Now both the HCM and our service show **7**.

If the HCM happens to be down at step 3, that's fine: we keep trying automatically until it succeeds. Alice and her manager never have to wait.

How holding days works (the details)

This is the heart of the service, so here is exactly how the numbers move.

Every balance is just **two numbers** plus one simple formula:

- **balanceDays** — the official total (what the HCM says, minus deductions we've confirmed).
- **pendingDays** — how many days are currently *on hold* by in-flight requests.
- **What you can still book** is the difference:

$$\text{available} = \text{balanceDays} - \text{pendingDays}$$

A *new* request is allowed only if it fits inside **available**. That single check is what guarantees we never overspend — and it happens **on our side, before the HCM is ever contacted**.

Here's what each event does to the two numbers:

Event	balanceDays	pendingDays	History entry
Employee creates a request (hold N days)	—	+N	HOLD
Manager rejects / cancels before filing	—	−N	RELEASE
HCM confirms the deduction	−N	−N	DEBIT
Cancel after it was already deducted (refund)	+N	—	CREDIT

Notice that a hold only ever touches **pendingDays**. The official **balanceDays** doesn't drop until the HCM **confirms** the deduction — at which point the hold is "converted" into a real debit (both numbers go down by N together).

Worked example — Alice, 10 days:

Step	What happens	balanceDays	pendingDays	available
Start	seeded from the HCM	10	0	10
Alice requests 3	3 days held	10	3	7
Manager approves	filing queued (no number change yet)	10	3	7
HCM confirms	hold becomes a real deduction	7	0	7

Every one of those steps also writes a line to the **ledger** (the history log), recording the change and the new totals. Because the ledger is append-only, we can always replay it to re-derive today's balance — and an automated test proves the replay always matches.

Keeping in step with the HCM (balance sync — the details)

Because the HCM can change balances on its own, we regularly read the HCM and update our copy. This is called **reconciliation**. There are three ways the HCM's numbers reach us, and they all run through the *same* logic:

1. **Scheduled pull — every 5 minutes.** We ask the HCM for the full list of balances and reconcile each one. This is the safety net: it keeps us correct even if nothing else happens.
2. **Push (webhook).** The HCM can notify us the moment something changes (faster than waiting 5 minutes). Same handling, whether it's one balance or the whole list.
3. **Targeted re-check.** Right after the HCM refuses one of our filings, we immediately re-read just that one balance to repair our view.

The core rule: for each balance, **the HCM number wins** for `balanceDays`, but we **keep our holds** (`pendingDays`) untouched — the HCM doesn't know about in-flight requests. So a bonus or a yearly reset is simply accepted, and nothing in flight is lost.

On top of that rule, three safeguards keep the sync correct:

Safeguard 1 — Ignore stale updates. Each balance remembers the timestamp of the last update it accepted (`hcmAsOf`). If an update arrives that is *older* than that, we ignore it. This stops a late, out-of-order message from undoing newer information.

Safeguard 2 — No accidental refunds (the timing window). A snapshot from the HCM was taken at a certain moment. If we confirmed a deduction *after* that moment, the snapshot won't include it yet — so blindly trusting the snapshot would hand those days back. We prevent this by checking the **ledger**: we add back any HCM-confirmed deductions/refunds that happened *after* the snapshot's time, using a precise millisecond timestamp stored on each ledger entry.

Example: Alice has 10. She uses 4, and the HCM confirms it — now she has **6**. A batch snapshot taken a moment *earlier* still says 10. Naïvely we'd reset her to 10 (a wrong refund!). Instead we compute: snapshot 10 + (the -4 deduction that came after it) = **6**. Correct.

Safeguard 3 — Flag, don't silently break. If, after syncing, someone's holds add up to *more* than their new official balance (`available` would be negative — e.g. the HCM dropped the balance below what people already reserved), we do **not** quietly change anyone's holds. We record a **DISCREPANCY** in the history log, raise a warning, and report it so a person can decide what to do.

Every sync also writes a **RECONCILE** entry to the ledger (and a **DISCREPANCY** entry if one was detected), so the whole process is fully auditable.

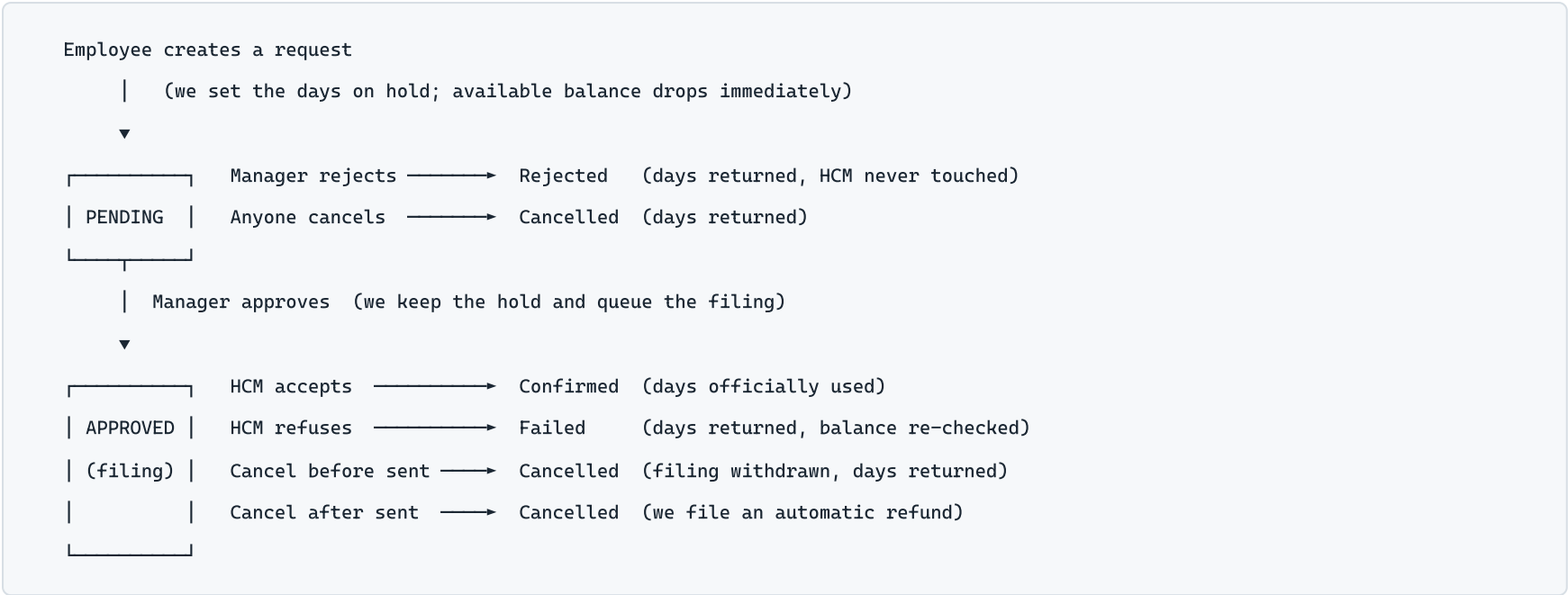
Reliable background work (so nothing gets lost)

When we need to tell the HCM something, we first **write it on a to-do list** (a database table called the *outbox*) in the very same step as the local change. A background worker then works through that list:

- **Success** → mark it done and confirm the deduction.
- **Temporary problem** (HCM busy or down) → try again later, waiting a little longer each time (about 1s, then 2s, then 4s...). After several failed tries we **park** it and alert a person — we never throw it away.
- **A firm "no" from the HCM** → release the hold, mark the request failed, and re-check the real balance.

Because the to-do item is saved **before** we call the HCM, even if our service crashes mid-call, the task is still there afterwards and gets finished. Each task carries a unique key, so if it ever gets sent twice it only counts once.

The life of a request



Column	Meaning
<code>approverId</code> , <code>decisionReason</code> , <code>failureReason</code> , <code>reason</code>	Who acted, and why.
<code>version</code> , <code>createdAt</code> , <code>updatedAt</code>	Bookkeeping.

`balance_ledger` — the append-only history. Rows are **never edited or deleted**.

Column	Meaning
<code>entryType</code>	What kind of change: <code>HOLD</code> , <code>RELEASE</code> , <code>DEBIT</code> , <code>CREDIT</code> , <code>RECONCILE</code> , <code>DISCREPANCY</code> , <code>ACCRUAL</code> .
<code>source</code>	What caused it (employee request, manager decision, HCM, reconciler...).
<code>balanceDelta</code> , <code>pendingDelta</code>	The signed change this entry made to each number.
<code>balanceAfter</code> , <code>pendingAfter</code>	The two totals right <i>after</i> this entry.
<code>requestId</code>	Which request it relates to (if any).
<code>occurredAtMs</code>	Exact event time, to the millisecond — used to order events correctly during sync.
<code>metadata</code>	Extra context (e.g. the raw HCM value, or a discrepancy reason).

`hcm_outbox` — the to-do list of messages we still owe the HCM.

Column	Meaning
<code>operation</code>	<code>FILE_DEBIT</code> (deduct) or <code>FILE_CREDIT</code> (refund).
<code>deltaDays</code>	The signed change to file (negative = deduction, positive = refund).
<code>idempotencyKey</code>	Sent to the HCM so a re-send is never double-counted.
<code>status</code>	<code>PENDING</code> → <code>RETRY</code> / <code>SENT</code> / <code>FAILED</code> / <code>DEAD</code> / <code>CANCELLED</code> .
<code>attempts</code> , <code>maxAttempts</code> , <code>nextAttemptAt</code>	Retry bookkeeping (how many tries, the limit, and when to try next).
<code>lastError</code> , <code>hcmReference</code>	The last failure message, and the HCM receipt once sent.

If something goes wrong (at a glance)

Situation	What the service does
You ask for more days than you have	Refuses instantly; the HCM is never called
You ask for an unknown person or place	Refuses with a clear "invalid" message
Bad input (e.g. a malformed date)	Refuses with a clear validation error
HCM is temporarily down when filing	Keeps retrying automatically; your hold stays safe

Situation	What the service does
HCM stays down after several tries	Parks the task and alerts a person — nothing is lost
HCM firmly refuses a filing	Returns the days, marks it failed, re-checks the real balance
A late/old update arrives from the HCM	Ignored, so it can't undo newer information
HCM shows fewer days than are reserved	Flagged for review — nothing changed silently
Two requests race for the last day	Only the ones that fit succeed; never an over-spend
The same request is sent twice	Counted once

Doing many things at once (no double-spending)

If 10 people try to grab from a balance of **5** at the exact same instant, **exactly 5 succeed and 5 are politely refused** — never 6. The balance can never go negative. (This is an actual automated test.) The `version` counter on each balance row is what makes this safe, and an accidental duplicate request is always counted only once.

What we deliberately left out (for now)

These are noted as "needed before real production," but were out of scope for this build:

- Logins and permissions, multiple companies, and rate-limiting.
- Weekend/holiday calendars — for now we count plain calendar days.
- A production-grade database setup and a dedicated message queue (the current design is ready to upgrade to these without rewriting the logic).

A quick look at the API

These are the web addresses the service offers, here for reference.

What you want to do	Request
Create a time-off request	<code>POST /v1/time-off-requests</code>
Look up one request	<code>GET /v1/time-off-requests/:id</code>
Approve / reject / cancel a request	<code>POST /v1/time-off-requests/:id/approve</code> (or <code>/reject</code> , <code>/cancel</code>)
Check someone's balance	<code>GET /v1/balances/:employeeId/:locationId</code>
Sync with the HCM now	<code>POST /v1/hcm/sync</code>
Receive a balance update from the HCM	<code>POST /v1/hcm/webhook/balances</code>
Health check	<code>GET /health</code>

How we know it works

The service has **67 automated tests, and they all pass**, covering about **92% of the code** (the most important logic is at 100%). The companion "**Testing in Plain English**" document lists, in everyday language, exactly what we check.